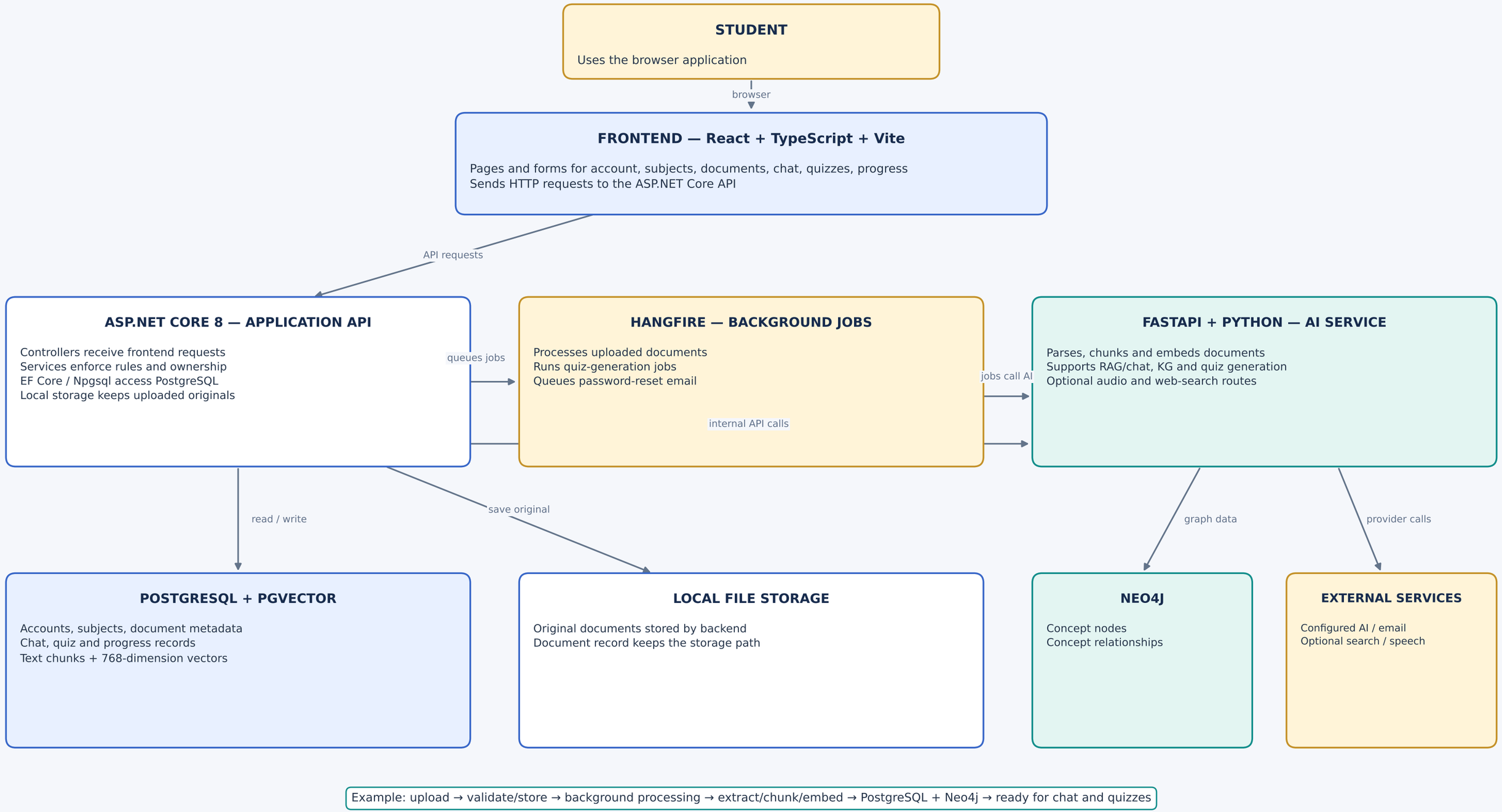
EduMind AI — System Architecture

Main application layers and verified data flow



Example: upload → validate/store → background processing → extract/chunk/embed → PostgreSQL + Neo4j → ready for chat and quizzes

EduMind AI — Core ER Diagram

Main PostgreSQL entities from schema.sql and EF Core models | PK = primary key | FK = foreign key

ACCOUNT + CONTENT

auth.users

PK id • email • password_hash • reset OTP

1 : N

content.subjects

PK id • FK user_id • name

1 : N

content.documents

PK id • FK user_id / subject_id • filename • status

1 : N

content.document_chunks

PK id • FK document_id
page range • embedding

1 : N

content.quiz_chunks

PK id • FK document_id
chunk + quiz metadata

CHAT + CITATIONS

rag.chat_sessions

PK id • FK user_id • title

1 : N

rag.chat_messages

PK id • FK session_id • role • content • kg_context

1 : N

rag.chat_web_sources

PK id • FK message_id • title • url

rag.chat_documents

PK/FK session_id
PK/FK document_id

rag.chat_citations

PK/FK message_id
PK/FK chunk_id

Bridge tables link sessions ↔ documents
and messages ↔ cited document chunks.

QUIZZES + ATTEMPTS

quiz.quizzes

PK id • FK user_id / subject_id • status

1 : N

quiz.quiz_questions

PK id • FK quiz_id
optional FK chunk_id

1 : N

quiz.quiz_options

PK id • FK question_id
label • option text

1 : N

quiz.quiz_attempts

PK id • FK quiz_id / user_id
score • attempt number

1 : N

quiz.quiz_answers

PK/FK attempt_id
PK/FK question_id

quiz.quiz_documents (bridge)

PK/FK quiz_id • PK/FK document_id

Answers refer to both the attempt and question.
Quiz documents are linked through a bridge table.

Neo4j concept nodes/edges are a separate graph store, not PostgreSQL tables. Supporting tables (notifications, activity logs, study plans) are omitted for readability.